

Multi-Tenant Inventory & Order Fulfillment System

Enterprise SaaS Prototype | Next.js 16 | TypeScript | Supabase | PostgreSQL | Tailwind CSS

Project Overview

The **Multi-Tenant Inventory & Order Fulfillment System** is an enterprise-grade Software-as-a-Service (SaaS) platform designed to manage inventory, warehouses, and order fulfilment for multiple independent businesses (tenants) within a single application.

Unlike traditional inventory management systems, this platform is architected around **tenant isolation**, **warehouse-level inventory tracking**, **transaction-safe stock operations**, and **concurrent order processing**.

Every tenant operates in a completely isolated environment where products, warehouses, inventory, and orders remain inaccessible to other organisations. Database-level Row-Level Security (RLS) ensures that isolation is enforced even if an application query is incorrect. This follows the requirements outlined in the project specification.

The system also introduces enterprise inventory concepts such as:

- Multi-warehouse inventory
- Atomic warehouse transfers
- Stock movement history
- Concurrent order reservation
- Inventory reconciliation
- Scheduled system audits

This project demonstrates the architecture and engineering practices used in production SaaS inventory platforms.

Project Goals

The primary objectives of this project are to:

- Build a scalable SaaS inventory management platform.
- Support multiple independent tenants.
- Maintain complete database-level tenant isolation.
- Manage inventory across multiple warehouses.
- Track every inventory movement.
- Prevent stock inconsistencies.
- Prevent overselling during simultaneous orders.

- Demonstrate enterprise software architecture.
 - Build using modern full-stack technologies.
 - Produce production-ready documentation.
-

Key Features

Multi-Tenant SaaS Architecture

- Tenant onboarding
 - Tenant-specific dashboards
 - Complete tenant isolation
 - Database-enforced Row-Level Security (RLS)
-

Warehouse Management

- Multiple warehouses per tenant
 - Warehouse CRUD operations
 - Warehouse-specific inventory
 - Warehouse dashboards
-

Product Management

- Product catalogue
 - SKU management
 - Product search
 - Categories
 - Product inventory linkage
-

Inventory Management

- Real-time inventory tracking
 - Per-warehouse stock management
 - Inventory adjustments
 - Stock history
 - Inventory audit trail
-

Stock Movement Tracking

Every inventory change generates a movement record.

Supported movement types include:

- Stock Received
- Stock Issued
- Manual Adjustment
- Warehouse Transfer
- Order Reservation

This creates an append-only audit trail for inventory changes, matching the movement-history model required by the project brief.

Warehouse Transfers

The system supports secure inventory transfers between warehouses.

Transfers are:

- Atomic
- Transaction-safe
- Logged
- Rollback protected

This ensures stock cannot be removed from one warehouse without being added to the destination warehouse.

Order Fulfilment

Features include:

- Order creation
 - Multiple order items
 - Inventory reservation
 - Order history
 - Order status tracking
-

Atomic Stock Reservation

One of the most important features of the application is preventing overselling.

The system uses database transactions so that:

- Competing users cannot purchase the same final unit.
- Inventory never becomes negative.
- Failed reservations automatically roll back.

The project specification explicitly requires database-enforced atomic reservation rather than application-level "check then write" logic.

Scheduled Reconciliation

Automated reconciliation jobs periodically verify inventory health by:

- Detecting low-stock items.
- Identifying inventory drift.
- Comparing movement history against current inventory.
- Creating reconciliation reports.

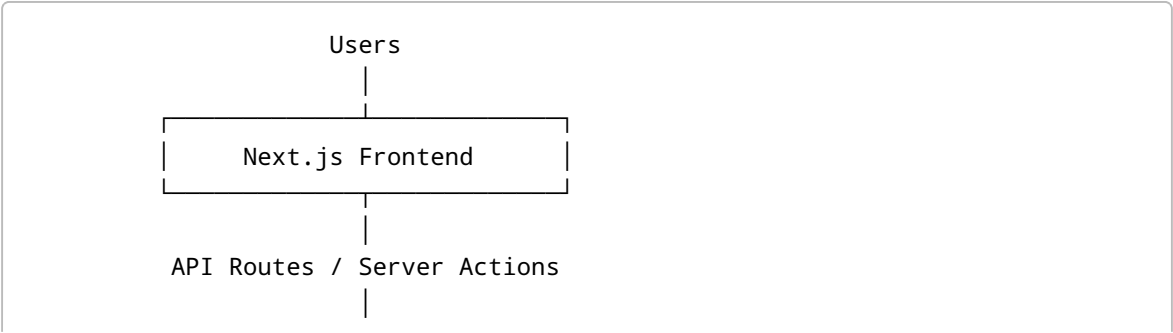
The reconciliation process is designed to be idempotent, meaning it can safely run multiple times without producing duplicate results, as required by the project brief.

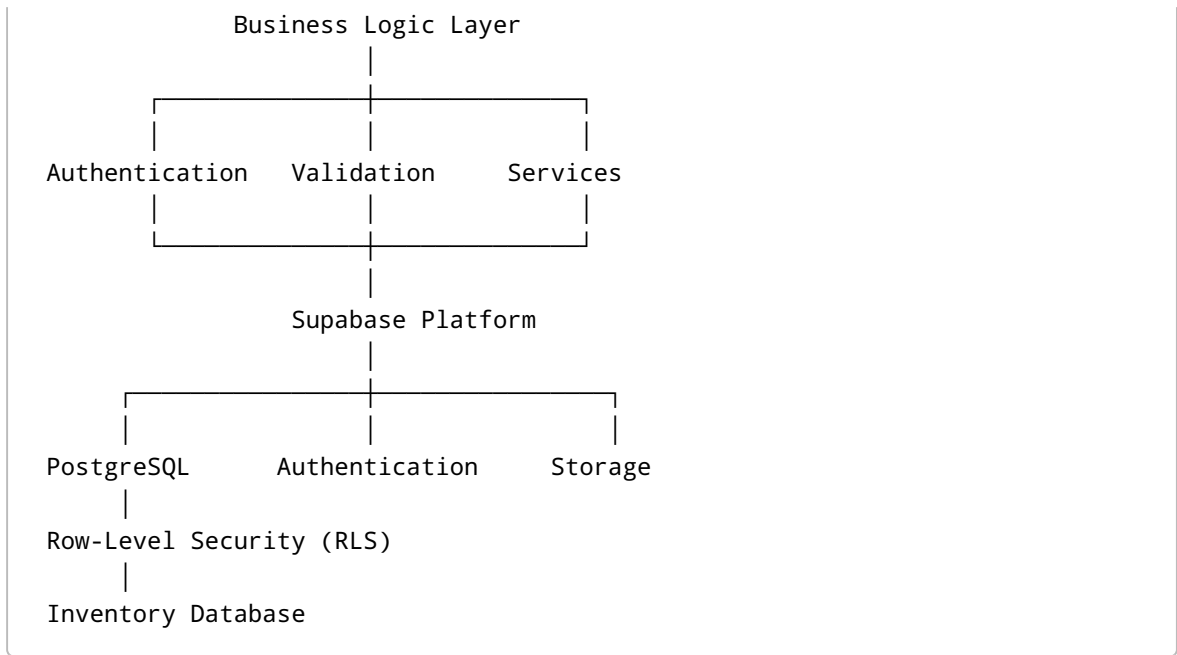
Technology Stack

Layer	Technology
Framework	Next.js 16.2
Language	TypeScript 6
Styling	Tailwind CSS v4
Backend	Next.js Server Actions
Validation	Zod
Authentication	Supabase Auth
Database	PostgreSQL 17
Database Platform	Supabase
Security	PostgreSQL Row-Level Security
Deployment	Vercel
Scheduled Jobs	Vercel Cron / Supabase Edge Functions

The selected technologies align with the recommended stack provided in the project specification.

High-Level Architecture





Project Structure

```
inventory-fulfillment-system/
├── app/
│   ├── auth/
│   ├── dashboard/
│   ├── inventory/
│   ├── warehouses/
│   ├── products/
│   ├── orders/
│   ├── transfers/
│   └── reconciliation/
├── components/
│   ├── ui/
│   ├── forms/
│   ├── layout/
│   ├── tables/
│   └── charts/
├── lib/
│   ├── auth/
│   ├── database/
│   ├── validation/
│   ├── services/
│   └── utils/
```

```
|─ actions/  
|─ hooks/  
|─ middleware/  
|─ types/  
|─ public/  
|─ styles/  
└─ docs/
```

Core Modules

The application is divided into modular business domains:

- Authentication
- Tenant Management
- User Management
- Warehouse Management
- Product Management
- Inventory Management
- Stock Movement Tracking
- Warehouse Transfers
- Order Management
- Inventory Reservation
- Scheduled Reconciliation
- Reporting & Analytics
- Administration

This modular architecture allows each business capability to evolve independently while sharing common infrastructure such as authentication, validation, and tenant isolation.

Development Roadmap (Overview)

The project is implemented in incremental phases:

1. Project Setup
2. Authentication
3. Multi-Tenant Architecture
4. Warehouse Management
5. Product Management
6. Inventory Management
7. Stock Movement Tracking
8. Warehouse Transfers
9. Order Management
10. Atomic Stock Reservation
11. Concurrency Testing
12. Scheduled Reconciliation
13. Analytics Dashboard

- 14. Administration Panel
- 15. UI/UX Refinement
- 16. Testing & Production Deployment

Each phase has a dedicated implementation blueprint and AI-agent prompt to ensure a structured, enterprise-grade development workflow.

Installation & Local Development

This section explains how to set up the project in a local development environment.

Prerequisites

Before running the application, ensure the following software is installed:

- Node.js 24 LTS
- npm / pnpm
- Git
- Supabase CLI
- PostgreSQL 17 (via Supabase)
- Visual Studio Code (Recommended)

Clone Repository

```
git clone https://github.com/your-username/multitenant-inventory-system.git  
  
cd multitenant-inventory-system
```

Install Dependencies

```
npm install
```

or

```
pnpm install
```

Configure Environment Variables

Create a `.env.local` file in the project root.

Example:

```
NEXT_PUBLIC_SUPABASE_URL=  
  
NEXT_PUBLIC_SUPABASE_ANON_KEY=
```



```
SUPABASE_SERVICE_ROLE_KEY=  
  
DATABASE_URL=  
  
NEXTAUTH_SECRET=
```

Note: Actual values should never be committed to source control.

Start Development Server

```
npm run dev
```

The application will be available at

```
http://localhost:3000
```

Database Architecture

The application uses **PostgreSQL 17** through **Supabase** as the primary database.

The database is designed around a **multi-tenant architecture**, where every tenant owns its own products, warehouses, inventory, and orders.

Database Philosophy

The database follows several important principles:

- Complete tenant isolation.
- Relational consistency.
- Transaction-safe updates.
- Immutable inventory history.
- Database-enforced security.
- High scalability.

Instead of relying on application filtering, every tenant-owned table contains a `tenant_id`, and PostgreSQL Row-Level Security (RLS) enforces access control. This approach aligns with the project brief's requirement for provable tenant isolation.

Database Tables

The core database consists of the following entities:

Tenant

Represents a business using the platform.

Stores:

- Business Name
 - Tenant ID
 - Created Date
 - Status
-

Users

Every authenticated user belongs to exactly one tenant.

Stores:

- User Information
 - Authentication Reference
 - Tenant Relationship
 - Role
-

Warehouses

Each tenant may own multiple warehouses.

Stores:

- Warehouse Name
 - Address
 - Tenant Reference
 - Status
-

Products

Stores inventory items.

Contains:

- Product Name
 - SKU
 - Category
 - Description
 - Tenant Reference
-

Inventory

Tracks stock for every product inside every warehouse.

Stores:

- Product
 - Warehouse
 - Current Quantity
 - Reserved Quantity
-

Stock Movements

An append-only history table.

Every inventory operation creates one movement record.

Movement Types:

- Receive
 - Issue
 - Adjustment
 - Reservation
 - Transfer
-

Orders

Stores customer orders.

Contains:

- Order Number
 - Status
 - Customer Reference (if implemented)
 - Tenant Reference
 - Creation Date
-

Order Items

Stores line items for every order.

Each record references:

- Order
- Product

- Quantity
 - Reserved Stock
-

Transfers

Represents warehouse-to-warehouse inventory movement.

Stores:

- Source Warehouse
 - Destination Warehouse
 - Product
 - Quantity
 - Transfer Status
-

Reconciliation Reports

Stores scheduled audit results.

Includes:

- Low Stock Alerts
 - Inventory Drift Flags
 - Timestamp
 - Resolution Status
-

Authentication Architecture

Authentication is handled using **Supabase Auth**.

The system supports secure authentication while ensuring every authenticated user belongs to exactly one tenant.

Authentication Flow

User

↓

Register

↓

Supabase Authentication

↓

User Profile Created

↓

Tenant Assigned

↓

Session Created

↓

Dashboard Access

Authentication Features

- Secure Login
- Secure Registration
- Logout
- Session Persistence
- Route Protection
- Server-side Authentication
- Client-side Session Handling

Multi-Tenant Architecture

This application is designed as a true Software-as-a-Service platform.

Instead of deploying a separate application for every business, all tenants use the same application while their data remains isolated.

Tenant Isolation Strategy

Every tenant-owned record contains:

tenant_id

Examples:

- Products

- Warehouses
- Inventory
- Orders
- Transfers
- Stock Movements

Database policies ensure users can only access records associated with their own tenant. The project specification explicitly requires Row-Level Security (RLS) to enforce this isolation.

Inventory Workflow

Inventory management follows an append-only model.

Current stock is derived from inventory operations while preserving a complete movement history.

Inventory Lifecycle

Product Created

↓

Inventory Added

↓

Warehouse Assigned

↓

Stock Received

↓

Stock Reserved

↓

Order Fulfilled

↓

Inventory Updated

↓

Movement Recorded

Inventory Principles

- Never delete movement history.
 - Every stock change is recorded.
 - Maintain full auditability.
 - Support warehouse-specific inventory.
 - Ensure inventory consistency.
-

Warehouse Transfer Workflow

Inventory can be transferred between warehouses belonging to the same tenant.

Transfers are performed as a single atomic transaction.

Transfer Lifecycle

Select Source Warehouse

↓

Select Destination Warehouse

↓

Validate Stock

↓

Begin Transaction

↓

Decrease Source Stock

↓

Increase Destination Stock

↓

Record Transfer

↓

Commit Transaction

If any operation fails, the transaction is rolled back to maintain consistency, reflecting the all-or-nothing requirement in the project brief.

Order Fulfilment Workflow

Order processing is tightly integrated with inventory management.

The workflow ensures inventory is reserved before an order can be confirmed.

Order Processing

Customer Order Created

↓

Validate Product Availability

↓

Reserve Inventory

↓

Create Order Items

↓

Update Inventory

↓

Generate Movement History

↓

Order Confirmed

Atomic Stock Reservation

One of the core engineering goals of the platform is preventing inventory overselling.

The project brief requires stock reservation to be handled atomically at the database level rather than through separate read-and-write operations.

Reservation Strategy

The reservation process follows this sequence:

Begin Database Transaction

↓

Attempt Reservation

↓

Inventory Available?

↓

YES → Reserve Stock

↓

NO → Rollback

↓

Commit Successful Reservation

This approach ensures:

- No negative inventory.
 - No duplicate reservations.
 - No race conditions.
 - Correct behaviour under concurrent requests.
-

Security Architecture

Security is implemented across multiple layers.

Application Security

- Secure authentication
- Protected routes
- Input validation

- Session management
-

Database Security

- Row-Level Security (RLS)
 - Tenant isolation
 - Foreign key constraints
 - Transactions
 - Indexed relationships
-

Validation

Every request entering the application is validated using **Zod** before business logic is executed.

Auditability

Every important inventory operation is recorded through movement history and transfer records, providing a complete audit trail for operational transparency.

Development Roadmap

The project follows a structured, phase-based development approach. Each phase builds upon the previous one to ensure modularity, maintainability, and production readiness.

Phase 0 — Project Setup

Objective: Establish the foundation of the application.

Deliverables

- Next.js 16 App Router setup
 - TypeScript configuration
 - Tailwind CSS v4 integration
 - Supabase project connection
 - Environment configuration
 - Enterprise folder structure
 - ESLint & Prettier configuration
-

Phase 1 — Authentication

Objective: Secure user authentication and session management.

Deliverables

- User registration
 - Login
 - Logout
 - Protected routes
 - Session management
 - Middleware
 - Authentication guards
-

Phase 2 — Multi-Tenant Architecture

Objective: Implement complete tenant isolation.

Deliverables

- Tenant onboarding
- Tenant creation
- Tenant assignment
- Row-Level Security (RLS)
- Tenant policies

- Tenant-aware queries
-

Phase 3 — Warehouse Management

Objective: Enable warehouse administration.

Deliverables

- Warehouse CRUD
 - Warehouse dashboard
 - Warehouse forms
 - Warehouse search
 - Warehouse management
-

Phase 4 — Product Management

Objective: Manage tenant-specific products.

Deliverables

- Product CRUD
 - SKU management
 - Product categories
 - Product search
 - Product filtering
-

Phase 5 — Inventory Management

Objective: Track inventory accurately.

Deliverables

- Inventory dashboard
 - Stock adjustments
 - Warehouse inventory
 - Current stock
 - Inventory summaries
-

Phase 6 — Stock Movement Tracking

Objective: Maintain a complete audit trail.

Deliverables

- Movement history

- Inventory timeline
 - Adjustment logs
 - Receive/Issue history
 - Inventory audit
-

Phase 7 — Warehouse Transfers

Objective: Move inventory safely between warehouses.

Deliverables

- Transfer creation
 - Transfer validation
 - Atomic transfer transactions
 - Transfer history
 - Rollback protection
-

Phase 8 — Order Management

Objective: Manage customer orders.

Deliverables

- Order creation
 - Order details
 - Order items
 - Order history
 - Order statuses
-

Phase 9 — Atomic Stock Reservation

Objective: Prevent inventory overselling.

Deliverables

- Transaction-safe reservation
 - Rollback handling
 - Inventory locking
 - Reservation logs
 - Stock validation
-

Phase 10 — Concurrency Testing

Objective: Verify behaviour under simultaneous requests.

Deliverables

- Multi-user testing
 - Stress testing
 - Race-condition validation
 - Reservation verification
 - Performance reports
-

Phase 11 — Scheduled Reconciliation

Objective: Detect inventory inconsistencies.

Deliverables

- Low stock reports
 - Inventory drift detection
 - Scheduled jobs
 - Reconciliation reports
 - Idempotent execution
-

Phase 12 — Analytics Dashboard

Objective: Provide operational insights.

Deliverables

- Dashboard KPIs
 - Charts
 - Reports
 - Warehouse statistics
 - Inventory analytics
 - Order analytics
-

Phase 13 — Administration Panel

Objective: Manage tenant configuration.

Deliverables

- Business settings
 - User management
 - Roles
 - Permissions
 - Profile management
 - Audit logs
-

Phase 14 — UI/UX Enhancement

Objective: Improve usability and accessibility.

Deliverables

- Responsive layouts
 - Loading skeletons
 - Empty states
 - Error pages
 - Dark mode
 - Animations
 - Accessibility improvements
-

Phase 15 — Testing & Deployment

Objective: Prepare the application for production.

Deliverables

- Unit testing
 - Integration testing
 - Production build
 - Performance optimisation
 - Deployment to Vercel
 - Final documentation
-

Testing Strategy

Testing is performed throughout the development lifecycle to ensure correctness, reliability, and security.

Functional Testing

- Authentication
 - Tenant onboarding
 - Warehouse operations
 - Product management
 - Inventory updates
 - Order processing
 - Warehouse transfers
-

Security Testing

- Row-Level Security validation

- Tenant isolation verification
 - API authorisation
 - Session validation
 - Input validation
-

Concurrency Testing

The project specification requires that concurrent orders never oversell inventory. Testing includes multiple simultaneous reservation requests to verify that only available stock is reserved and failed requests receive a clear insufficient-stock response.

Performance Testing

- Large inventory datasets
 - Multiple warehouses
 - Concurrent users
 - Database optimisation
 - Dashboard responsiveness
-

Deployment

The application is intended to be deployed using:

Frontend

- Vercel

Backend

- Next.js Server Actions
- API Routes

Database

- Supabase PostgreSQL

Authentication

- Supabase Auth

Scheduled Jobs

- Vercel Cron
 - Supabase Edge Functions
-

Future Enhancements

The current prototype intentionally focuses on inventory operations. Future enhancements may include:

- Barcode & QR code scanning
- Purchase order management
- Supplier management
- Customer management
- Batch and lot tracking
- Expiry date tracking
- Email notifications
- Mobile application
- Offline-first support
- Business intelligence dashboards
- AI-powered demand forecasting
- Multi-language support
- Multi-currency support
- WebSocket-based real-time updates
- ERP integrations
- Shipping and courier integrations
- Payment processing
- Subscription billing

These features are outside the current project scope and are listed as potential future improvements.



Contributing

Contributions are welcome.

To contribute:

1. Fork the repository.
 2. Create a feature branch.
 3. Implement your changes.
 4. Follow the project's coding standards.
 5. Test all functionality.
 6. Submit a Pull Request with a detailed description of the changes.
-

Documentation

Project documentation consists of three primary documents:

- **Software Architecture Document (SAD)** – overall system design and architecture.
- **AI Development Blueprint** – phase-by-phase implementation prompts for the AI coding agent.
- **README** – project overview, setup instructions, workflows, and roadmap.

Project Roadmap

```
graph TD; A[Project Setup] --> B[Authentication]; B --> C[Multi-Tenant Architecture]; C --> D[Warehouse Management]; D --> E[Product Management]; E --> F[Inventory Management]; F --> G[Stock Movement Tracking]; G --> H[Warehouse Transfers]; H --> I[Order Management]; I --> J[Atomic Stock Reservation]; J --> K[Concurrency Testing]; K --> L[Scheduled Reconciliation]; L --> M[Analytics Dashboard]; M --> N[Administration Panel]; N --> O[UI/UX Enhancement]; O --> P[Testing]; P --> Q[Production Deployment];
```

Learning Outcomes

This project demonstrates practical implementation of:

- Enterprise SaaS Architecture
 - Multi-Tenant System Design
 - PostgreSQL Row-Level Security
 - Secure Authentication
 - Warehouse Inventory Management
 - Transaction Management
 - Atomic Database Operations
 - Concurrent Request Handling
 - Inventory Reconciliation
 - Modern Full-Stack Development
 - Scalable Software Architecture
 - Production Deployment
-

License

This project is intended for educational, portfolio, and prototype purposes.

You may modify and extend the project to suit your own requirements. Commercial use should ensure compliance with the licenses of all third-party libraries and services.

Acknowledgements

This project is based on the "**Multi-Tenant Inventory & Order Fulfillment (Prototype)**" screening brief, with the architecture, implementation plan, and documentation expanded into an enterprise-style software specification. The implementation follows the functional requirements, non-functional requirements, and recommended technology stack described in the original brief.

Conclusion

The **Multi-Tenant Inventory & Order Fulfillment System** is designed as a comprehensive demonstration of modern SaaS engineering principles. By combining secure multi-tenancy, warehouse-level inventory tracking, atomic order reservation, scheduled reconciliation, and a modular architecture, the project showcases how enterprise inventory platforms can be built using Next.js, TypeScript, Supabase, PostgreSQL, and Vercel.

The accompanying Software Architecture Document and AI Development Blueprint provide a structured path from initial setup to production deployment, making the project suitable as a learning resource, portfolio showcase, and foundation for future expansion.